FaceLit — base de datos y migraciones

Motor de referencia: PostgreSQL 16 con extensión pgvector.

Estado reportado al 27 de octubre de 2026: 45 changesets aplicados en producción después de la actualización documentada en Hostinger.

Revisión complementaria: 4 de noviembre de 2026. Se inspeccionó el árbol local de `FaceLit-DB` y el modelo legal del backend. Los archivos de migración 025 están sin seguimiento en Git y su changelog aún no aparece incluido en `01_ddl/04_alter/0000changelog.yaml`; por tanto, no se consideran parte de la cadena activa ni aplicados en producción. El estado de 45 changesets corresponde al reporte del 27 de octubre.

Modelo lógico

El esquema se divide en siete áreas:

Esquema	Entidades/propósito
security	Usuarios, credenciales, sesiones, recuperación y configuración
roleandpermission	Roles, permisos y sus asignaciones
academic	Programas, fichas, aprendices, instructores y relaciones
environment	Ambientes y sesiones de asistencia
facialrecognition	Referencias faciales, eventos, dispositivos y solicitudes de actualización
notification	Notificaciones y registro de correo
legal	Aceptación de términos y aviso de privacidad

Los scripts de creación identifican 31 tablas de aplicación; Liquibase mantiene además sus propias tablas de control. Entidades centrales: user_app, credential, role, permission, program, chip, apprentice, instructor, apprentice_chip, instructor_chip, environment, record_environment, user_face y facial_event.

Claves foráneas preservan relaciones; tablas de asociación evitan repetir datos de personas/fichas; UUID identifica registros; índices/restricciones apoyan unicidad y consultas. Ninguna de esas decisiones reemplaza la autorización de aplicación.

En el esquema DDL actualmente enlazado, `legal` contiene `terms_acceptance`. El backend local también presenta entidades `Consent`, `ConsentVerification` y `Guardian`, pero en la cadena de migraciones inspeccionada no se encontró un changeset que cree las tablas correspondientes. Antes de integrar ese flujo, debe verificarse que el esquema desplegado incluya esas tablas y claves foráneas; las entidades JPA por sí solas no las crean porque el backend valida el esquema.

Evolución H-50: nomenclatura inglesa

El backend actualizado requería nombres de columnas en inglés, mientras que la base anterior tenía nombres históricos. El trabajo documentado añade y aplica una evolución de esquema coordinada:

Nombre anterior	Nombre canónico
roleandpermission.role.name_rol	role_name
roleandpermission.permission.descripcion	description
roleandpermission.user_role.asigned_at	assigned_at
fk_facial_event_apprendice	fk_facial_event_apprentice
SEGURIDAD / ACADEMICO / ASISTENCIA / RECONOCIMIENTO_FACIAL / TRASLADO	SECURITY / ACADEMIC / ATTENDANCE / FACIAL_RECOGNITION / TRANSFER

Cómo se migró

• **Expansión:** agregar columnas canónicas en inglés, copiar datos de columnas históricas, mantener ambas nomenclaturas durante la transición y sincronizar escrituras con triggers.

• **Compatibilidad de consumidores:** actualizar mapeos JPA, consultas/catálogos de notificación y consumidores; el frontend acepta categorías antiguas y nuevas durante el cambio.

• **Contracción:** retirar triggers/columnas antiguas cuando los escritores ya usan el contrato nuevo, normalizar valores históricos, renombrar la FK y aplicar restricción CHECK sobre categorías inglesas.

• **Reversión:** mantener rollback para cada etapa y validar rollback/reaplicación en una base PostgreSQL desechable. No editar los changesets históricos ya aplicados.

La inclusión de las migraciones H-50 en FaceLit-DB/main se hizo mediante PR #63, con CI obligatorio de PostgreSQL/Liquibase según los informes del equipo. El backend nuevo requiere al menos la expansión del esquema antes de su inicio; la contracción requiere haber detenido o actualizado escritores antiguos.

Aceptación por versión del aviso (cambio local 025)

El archivo local `01_ddl/04_alter/025_terms_acceptance_notice_version.sql` agrega `notice_version` a `legal.terms_acceptance`, asigna a las aceptaciones anteriores una versión `legacy:<id_acceptance>` y crea el índice único `uq_terms_acceptance_user_notice_version` sobre usuario y versión. Su rollback elimina el índice y la columna. El changelog declara el changeset `facelit-terms-acceptance-notice-version`, pero todavía no está incluido en el changelog padre de alteraciones y sus archivos no están confirmados en Git. Por ello, el cambio no se ejecuta con la cadena activa hasta que se integre explícitamente y se valide en una base desechable.

La nueva columna es no nula y el modelo JPA local la marca obligatoria. La implementación del servicio de aceptación también debe persistir la versión recibida/configurada; confirmar esa correspondencia antes de ejecutar la migración o arrancar el backend con validación de esquema.

Despliegue de esquema y limpieza de datos

El informe de actualización registra que producción estaba en 39 changesets antes del despliegue. Se generó un respaldo custom, se validó el changelog, se detuvieron los servicios, se creó un respaldo consistente previo a la activación y Liquibase aplicó las migraciones hasta 45, sin pendientes.

Luego se realizó una limpieza transaccional autorizada para preparar la demostración. Se preservaron cuenta, credencial, roles/permisos, configuración y aceptación legal del coordinador; se eliminaron registros operativos anteriores. Estado posterior reportado: 1 usuario, 1 credencial, 3 roles, 12 permisos y cero instructores, aprendices, programas, fichas, referencias faciales y eventos. Los datos locales no se importaron a producción.

En la base local de verificación H-50 también se compararon huellas normalizadas de roles, permisos, asignaciones y notificaciones antes/después. Ese resultado corresponde a la base local de prueba y no debe confundirse con el estado limpio de producción.

Cómo aplicar cambios en desarrollo

En FaceLit-DB se usan Docker Compose y Liquibase. Antes de modificar una base, confirmar destino, respaldo y estado de changelog. Secuencia documentada:

docker compose -p facelit-docker-compose up -d postgres

docker compose -p facelit-docker-compose --profile tooling build liquibase

docker compose -p facelit-docker-compose --profile tooling run --rm liquibase validate

docker compose -p facelit-docker-compose --profile tooling run --rm liquibase status

docker compose -p facelit-docker-compose --profile tooling run --rm liquibase update-sql

docker compose -p facelit-docker-compose --profile tooling run --rm liquibase update

Los comandos dependen de los servicios/perfil del Compose de FaceLit-DB. Revisar el archivo de Compose de la rama utilizada; el despliegue Hostinger tiene un Compose coordinador distinto. No usar una configuración de desarrollo contra

producción.

Validaciones reportadas y límites

Los informes documentan pruebas de expansión, copia de datos, compatibilidad de escritores, restricciones, ambas reversiones y reaplicación sobre base desechable. La actualización de producción pasó validación y dejó 45 changesets sin pendientes.

Esto no acredita que todos los rollback históricos estén probados, que todos los backups hayan sido restaurados, ni que las tablas sensibles estén sujetas a una política completa de retención. La restauración se mantiene como pendiente de validar en un entorno separado.

Fuentes técnicas

FaceLit-DB/changelog-master.yaml, 01_ddl/, 02_dml/ y 05_rollbacks/. Para el cambio local 025: `01_ddl/04_alter/025_terms_acceptance_notice_version.changelog.yaml`, su SQL y rollback. La consolidación DDL/DML/DQL histórica debe cotejarse contra toda la cadena de cambios; un diccionario basado solo en CREATE TABLE no incorpora automáticamente ALTER TABLE posteriores.